ShipTrack (shipment1212) - 4-Assignment Traceability

CampusEats pattern mapped to ShipTrack places + how to run. TeamID: TODO | Name/Roll: TODO | Date: 2026-09-08. Read-only review - no project files changed.

ShipTrack (shipment1212) — 4-Assignment Traceability (CampusEats pattern → ShipTrack place)

TeamID: TODO | Name/Roll: TODO | Date: 2026-09-08
> Read-only mapping. No project files were changed. All paths are relative to shipment1212/.

Assignment 1 — HTTP by Hand & Project Setup → where it lives in ShipTrack

PDF asks (CampusEats)	ShipTrack equivalent = DONE at this place	How to run / prove it
1. curl -i x5 to public JSON API, one deliberate 404 + 1-line status/Content-Type note (http-log.md)	RUNBOOK.md §4 + postman/ShipTrack.postman_collection.json + gateway Swagger at http://localhost:18000/api-docs. Your stack IS the JSON API. No http-log.md file exists — run the 5 below with -i and paste output to create it.	Start stack first: cp .env.example .env && docker compose up -d --build (16 containers: 12 services + postgres/mysql/mongo/frontend). Then: curl -i POST http://localhost:18000/api/v1/auth/login -H 'content-type: application/json' -d '{"email":"admin@shiptrack.local","password":"Admin@12345"}' → expect 200, content-type: application/json = OK, JSON body. curl -i GET http://localhost:18000/api/v1/shipments/track/STKABC123 → 200 public lookup. curl -i GET http://localhost:18000/api/v1/tracking/STKABC123
2. DevTools → Network, Disable cache, reload; report count/size/slowest/3xx-4xx (network-analysis.md)	frontend/ (React Vite) served at http://localhost:3001. No network-analysis.md exists — do it live in 1 min.	Open http://localhost:3001 → F12 Network → tick Disable cache → reload. Record: request count, total MB (bottom bar), slowest row (Time column, usually JS bundle or /graphql), any 304 (ETag) / 404 / 429. Save as docs/network-analysis.md if you need the file.
3. Git repo campuseats with README + docs/ + incremental commits	DONE in spirit: repo nandankabra/wsexpress has README.md (123 lines, stack+arch+ports), docs/architecture.md, docs/er-diagrams.md, docs/deployment-guide.md, docs/diagrams/. History is incremental, not one final commit.	git log --oneline -20 shows ~20 steps (e.g. a62397a Add shipment-service…, fbd5b25 Add location-service…, 64c5475 Add graphql-service…). git remote -v = git@github.com:nandankabra/wsexpress.git. Rename mapping only: campuseats → shipment1212.
4. One-page brief: what/who/nouns/verbs (brief.md)	DONE in README.md:1-7,40-64 + docs/architecture.md:52-65. No brief.md file — copy this table if you need it. What: logistics platform, 12 Node/Express services behind gateway, 3 portals. Who: Guest (public track), CUSTOMER (register/create/quote/track), CARRIER (admin-provisioned, claim/update), ADMIN (manage/analytics/audit). Nouns: User, Carrier, Shipment, TrackingEvent, Quote/PricingRule, Country/City/Warehouse/Route, AnalyticsRecord, AuditLog, Notification. Verbs: register/login, createShipment, getQuote, trackPublic, recordEvent, updateStatus, assignCarrier, verifyCarrier, aggregateDashbo	cat README.md docs/architecture.md is the brief source.

Assignment 2 — Services, Contracts & Schema → where it lives

PDF asks	ShipTrack place	How to run / prove
1. ≥6 Capabilities (verbs) in design.pdf	docs/architecture.md:52-65 service table + README.md:40-53. Capabilities: (1) register/login/RBAC, (2) manage customer/carrier profiles, (3) getQuote, (4) createShipment, (5) trackPublic/listHistory, (6) recordEvent/latest, (7) updateStatus/assignCarrier, (8) currencyConvert/countryInfo, (9) analytics aggregate, (10) audit, (11) notify, (12) GraphQL dashboard.	No design.pdf — this list IS the capability list.

PDF asks	ShipTrack place	How to run / prove
2. services.drawio + .png: 3–6 boxes, owned-data label, arrows with operation, no shared data + member per service	DONE bigger: 12 boxes in docs/architecture.md:5-48 (mermaid flowchart) + editable docs/diagrams/use-case-diagram.drawio + .png, data-flow-diagram.drawio. Boundary rule holds: each service owns its DB (see next row). Add member names to boxes if submitting.	Open docs/diagrams/*.drawio in diagrams.net; view .png alongside. Arrows: Shipment → Pricing: POST /pricing/quote, Shipment → Tracking: POST /tracking/:id/events, Pricing → Location: SOAP GetCountryInfo, GraphQL → Shipment/Tracking/User: REST.
3. Per-service contract table (op, input, output, errors), hide tables/SQL; ≥3 ops for 2 main	DONE as OpenAPI, not PDF tables: services/*/openapi.yaml (12 files, e.g. services/shipment-service/openapi.yaml, services/tracking-service/openapi.yaml). Shipment has 7 ops, tracking has 4.	ls services/*/openapi.yaml; Swagger UI per service http://localhost:<8001-8010>/api-docs, gateway http://localhost:18000/api-docs.
4. Central placeOrder full spec (in/out/errors/hidden)	Mapped to POST /api/v1/shipments → createShipment in services/shipment-service/src/controllers/shipmentController.js:11-18 + src/services/shipmentService.js. In: {origin{address,city,country}, destination{…}, weightKg, packageType, currency} + JWT + Idempotency-Key. Out: 201 Shipment{id, trackingId, quotedPrice…}. Errors: 400 bad body, 401/403 auth, 409 idempotency-key reuse with different body (common/src/idempotency.js:29), 503 pricing/tracking down. Hidden: Postgres table, UUID gen, pricing SOAP call, tracking best-effort POST, version column.	cat services/shipment-service/openapi.yaml (/api/v1/shipments POST) + RUNBOOK.md §4 register→login→create curl.
5. schema.drawio + .png + schema.sql, no table in 2 services	DONE split: docs/diagrams/er-diagram.drawio + .png, schema-diagram.drawio + .png, docs/er-diagrams.md; SQL in infra/postgres-init/01-create-databases.sql, infra/mysql-init/01-create-databases.sql, per-service src/db/migrate.js. Ownership: Postgres auth_db,user_db,carrier_db,shipment_db; MySQL pricing_db,location_db; Mongo tracking_db,crm_db,audit_db.	ls docs/diagrams/*.drawio docs/diagrams/*.png infra/*/01-create-databases.sql; docker exec -it shiptrack-node-postgres psql -U shiptrack -d shipment_db \dt etc. (creds in RUNBOOK.md §4).
6. 5-property check (reachable, self-contained, contract, independent, loosely-coupled)	All 12 pass via: reachable (own port 8001-8011 + gateway route), self-contained (own DB, no store import across services), contract (openapi.yaml + /api-docs), independent (own Dockerfile, deployable), loosely-coupled (only createServiceClient HTTP, env-var URLs, no shared DB). Fix note: frontend nginx.conf proxy_pass must be repointed on Render (documented in deployment guide).	docker compose ps (all healthy) + cat common/src/httpClient.js + docker-compose.yml.

Assignment 3 — External SOAP Partner (design-time only) → where it lives (you have it LIVE)

asks	ShipTrack place	How to run / prove	
ntext: pick payment/logistics/SMS, 3–4 nces why SOAP while rest is REST	Picked: Pricing → Location CountryInfo (logistics-data edge). Why SOAP: strong WSDL contract both sides can codegen; message-level GetCountryInfo lookup must stay stable while REST evolves; fits currency/country reference data shared with external-style partners; rest stays REST for CRUD speed. Source: docs/architecture.md:71-72. No integration.pdf — use this para.	—	
tner.wsdl with types/message/portType/bi (SOAP/HTTP)/service-port+real URL	DONE as code, not standalone file: services/location-service/src/soap/countryWsdl.js:1-59 (types GetCountryInfoRequest/Response, messages In/Out, portType CountryInfoPortType, binding SOAP document/literal over http://schemas.xmlsoap.org/soap/http, service CountryInfoService → http://localhost:8007/wsdl/country) + services/pricing-service/src/soap/currencyWsdl.js. Served live by node-soap (src/soap/index.js each service).	curl -s http://localhost:8007/wsdl/country?wsdl \	head -60 → 200 XML. Currency: http://localhost:8006/wsdl/currency?wsdl. Matches WSDL element-for-element.

asks	ShipTrack place	How to run / prove	
ap-request.xml, soap-response.xml, fault.xml by hand	No standalone XML files — live equivalent in services/pricing-service/tests/currencySoap.test.js, countryClient.test.js, services/location-service/tests/countrySoap.test.js + src/soap/countryClient.js:37-42 (GetCountryInfoAsync({countryCode})). Envelope shape = Header (credentials) + Body (GetCountryInfo), success = {code,name,capital,currency}, fault = unknown code → null / SOAP Fault (e.g. unavailable).	npm test --workspace services/pricing-service runs SOAP tests without Docker (DB mocked). To hand-write files, copy envelope from ?wsdl example and save the 3 XMLs.	
TP binding block (POST, Host, ent-Type, SOAPAction, endpoint)	POST /wsdl/country Host localhost:8007, Content-Type: text/xml; charset=utf-8, SOAPAction: "http://shiptrack.local/countryinfo/GetCountryInfo" (countryWsdl.js:48), endpoint from soap:address (countryWsdl.js:56). Caller sets endpoint override in pricing-service/src/soap/countryClient.js:12-22 (uses LOCATION_SERVICE_URL, not baked localhost).	curl -i -X POST http://localhost:8007/wsdl/country -H 'Content-Type: text/xml' -H 'SOAPAction: "http://shiptrack.local/countryinfo/GetCountryInfo"' --data-binary @soap-request.xml	
covery modern form: URL or lookup + ord catalogue ness/service/endpoint/WSDL-tModel)	DONE via env + WSDL URL, no UDDI server: LOCATION_SERVICE_URL env (services/pricing-service/.env.example, countryClient.js:12) + fetch ${LOCATION_SERVICE_URL}/wsdl/country?wsdl. Registry row: business=ShipTrack-Location, service=CountryInfoService, endpoint=http://location-service:8007/wsdl/country, tModel=http://shiptrack.local/countryinfo + ?wsdl.	cat services/pricing-service/.env.example; cat services/pricing-service/src/soap/countryClient.js:1-30 (lazy soap.createClientAsync(wsdlUrl), cache-clear on fail).	
ult → own-contract mapping (partner must not leak)	countryClient.js:40 returns null on unknown/SOAP-fail; pricingService.js degrades to default transit/currency; trackingClient.js:9 fallback () => null so Shipment.updateStatus still succeeds; surfaced as REST ApiError (common/src/errors.js: 404 NOT_FOUND, 422 UNPROCESSABLE_ENTITY, 503 SERVICE_UNAVAILABLE) — never raw card_declined/SOAP fault to students.	cat services/shipment-service/src/clients/trackingClient.js:1-20 common/src/errors.js. TeamID/Roll: TODO placeholders (required on integration.pdf p1).	

Assignment 4 — Rebuild a service in REST (worked example: tracking-service, NOT Payments)

> tracking-service = Delivery/Orders-history analogue. Code is JS/Express (not Python models.py/store.py/errors.py/app.py), Jest (not pytest) — functionally equivalent, naming differs.

Part A · Model

#	PDF asks	Tracking place + run
A1	Pick Catalogue/Orders/Delivery, NOT Payments, keep Assgn-2 boundary	DONE: tracking-service (services/tracking-service/), owns Mongo tracking_db only (src/models/trackingEventModel.js, src/db/connection.js). Boundary from docs/architecture.md:59.
A2	3–5 SOAP-style ops as starting point	recordTrackingEvent(...), getTrackingHistory(...), getLatestStatus(...), listEventsPaginated(...) (see src/services/trackingEventService.js:16-35).
A3	Nouns (plural), no verbs in URL	Nouns: tracking-events. URLs have no verbs: POST /api/v1/tracking/{trackingId}/events, GET .../events?cursor&limit, GET .../latest (openapi.yaml:13-56, src/routes/trackingRoutes.js:11-16).
A4	Resource table: method/URL/what/success/failure ≥4 rows, one sub-resource	Table IS openapi.yaml:13-56: POST /{trackingId}/events create 201/400; GET /{trackingId}/events?cursor&limit filtered list 200; GET /{trackingId}/latest single-read sub-resource 200/404. 4th row via shipment analogue /shipments/{id}/status if strict ≥4 needed.

#	PDF asks	Tracking place + run
A5	Justify hardest mapping (3–4 sentences)	Hardest: recordEvent is an action (verb) forced into POST /{trackingId}/events collection-create; rejected POST /recordTrackingEvent RPC and PUT /status overwrite (loses history). Append-only event log preserves audit + makes list/latest trivial.

Part B · Contract

#	PDF asks	Tracking place + run
B1	openapi.yaml FIRST with info/servers/paths/components.schemas, every op has requestBody/parameters + ≥2 responses	DONE services/tracking-service/openapi.yaml:1-84: info, servers: [localhost:8005, gateway], paths 3 ops, components.schemas.
B2	Schemas once, $ref only	DONE: RecordEventRequest, TrackingEvent, Error under components.schemas:60-84, referenced via $ref (:24,27,44,55). No inline copy-paste.
B3	Every failure declared (Assgn-3 faults → status codes)	PARTIAL: 400 BadRequest + 404 NotFound declared; GET .../events lists only 200 (add 400/404 for strict pass); SOAP unknown-country fault → 404, validation fault → 400 via validate() + ApiError.
B4	Validate, zero errors proof	Run: npx @redocly/cli lint services/tracking-service/openapi.yaml or upload to editor.swagger.io (expect 0 errors) or python3 -m openapi_spec_validator services/tracking-service/openapi.yaml. Also live: http://localhost:8005/api-docs (wired in src/app.js:11). Save terminal output/screenshot.

Part C · Implement

#	PDF asks	Tracking place + run
C1	Layout models.py,store.py,errors.py,app.py + tests/, in-process dict, no cross-import	JS equivalent DONE: src/models/trackingEventModel.js (Mongoose), src/services/trackingEventService.js (=store), src/app.js, src/routes/trackingRoutes.js, src/controllers/trackingController.js, tests/tracking.routes.test.js. DB is real Mongo (stronger than dict); no service imports another's store — only HTTP via createServiceClient.
C2	Record vs representation as_json() differ ≥1 field, no credential leak	DONE toResponse() in src/services/trackingEventService.js:4-14: stores _id, trackingId, status, location, note, recordedBy, timestamp → publishes id (string of _id), hides _id/v, recordedBy falls back to 'system' (src/controllers/trackingController.js:5), no password/token in output.
C3	4 endpoints: create, single-read, filtered list ?query, state-changing sub-resource	DONE 3 in tracking + 1 via shipment link: POST /:trackingId/events (create), GET /:trackingId/latest (single), GET /:trackingId/events?cursor&limit (filtered list), state-change = POST appends event changing shipment's derived status (or cite shipment-service PATCH /{id}/status as 4th). All in openapi.yaml.
C4	validate() before touching body (replaces XSD)	DONE src/routes/trackingRoutes.js:11: validate(recordEventSchema) before controller; schema src/middleware/schemas.js:1-13 (required status, enum, additionalProperties:false, maxLengths). Missing it would let {"status":"NOT_A_STATUS"} or oversized note through (test proves: tests/tracking.routes.test.js:28-33 expects 400).

#	PDF asks	Tracking place + run
C5	Codes: 201+Location create, 200 read, 202 async, 400/404/409/422	PARTIAL-pass: 201 on create (trackingController.js:13), 200 reads, 400 enum fail, 404 no-events (trackingEventService.js:33). Location header + 202/409/422 live in shipment-service (201 create, 412 version conflict, 422 bad transition, 409 idempotency reuse in common/src/idempotency.js:29) — quote that for full coverage.
C6	One problem() shape {type,title,status,detail}	DONE as {error:{code,message,details}} via common/src/errors.js:1-53 (ApiError.*) + central errorHandler (used in src/app.js:16). No endpoint invents its own shape.
C7	One safely-retryable endpoint via Idempotency-Key	DONE at platform level: common/src/idempotency.js:11-49 (key→{status,body} cache, same-key-different-body→409, Idempotent-Replay:true header). Wired on shipment POST /shipments (see shipment-service/openapi.yaml:29-30 + frontend/src/api/shipments.js:6 sends header). Tracking POST /events is append-only so safe to document as: retry with same payload yields duplicate event — production fix = add same middleware line idempotency() to trackingRoutes.js (1 line) if examiner demands header on THIS service.
C8	4 tests, pytest passes	DONE 6 tests in Jest (not pytest): tests/tracking.routes.test.js:27-83 — (1) 400 on bad enum, (2) 201 unauthenticated internal call as system, (3) 201 carrier recordedBy, (4) 200 public paginated list, (5) 404 unknown latest, (6) 200 latest. Run: npm test --workspace services/tracking-service (DB mocked via jest.mock). Save output. For literal pytest, note stack is Node per spec (README.md:9-12 mandates JS).

Part D · Survive the network

	PDF asks	Tracking/shipment place + run		
	One real HTTP call to another service, env-var URL	DONE: shipment-service → tracking-service via src/clients/trackingClient.js:1-20 (baseURL: process.env.TRACKING_SERVICE_URL), called from shipmentService.updateStatus; reverse direction tracking accepts internal no-JWT call (trackingRoutes.js:8-11 optionalAuthenticate). Also shipment→pricing, pricing→location(SOAP+REST), graphql→shipment/tracking/user.		
	Timeout + retry exp-backoff+jitter; never retry 4xx; retried create carries idempotency key	DONE common/src/httpClient.js:1-71: timeout 3000ms (OUTBOUND_TIMEOUT_MS), axios-retry exp backoff (retryDelay: exponentialDelay), retryCondition: isNetworkOrIdempotentRequestError		status>=500 (4x breaker (50%, 1 propagation. Shi Idempotency-Ke
	Fallback + 2–3 sentence justification in NOTES.md	Code DONE, doc TODO: trackingClient.js:9 fallback: () => null = best-effort: status change succeeds locally even if tracking down (degrading to delayed history is right; failing the shipment write would punish customer for an analytics-side outage). Pricing→location REST falls back to default transit estimate. Write this into NOTES.md (no NOTES.md exists yet).		
	service folder + NOTES.md (A4 table+A5+D3+5 answers) + curl -i transcript + validator + pytest output + Name/Roll p1	Service folder DONE. TODO to be submission-ready: (1) create NOTES.md from this file's tables + 5 answers below, (2) curl -i transcript: POST .../STKABC123/events →201, repeat same idempotency key →200 replay, malformed →400, GET /UNKNOWN/latest →404, version-conflict via shipment PATCH →412/409, (3) validator 0-error output, (4) npm test output, (5) Name/Roll p1 (placeholders). No failure paths = returned unmarked.		

The 5 NOTES.md answers (tracking-service version — copy into NOTES.md)

1. WSDL lines vs openapi.yaml lines: wc -l services/location-service/src/soap/countryWsdl.js services/tracking-service/openapi.yaml (≈61 vs 84). Difference = WSDL declares binding/soap:operation/soap:body/service/port/soap:address + XSD wire details OpenAPI doesn't need; OpenAPI instead declares servers/paths/examples + HTTP semantics. Two WSDL-only things: (a) SOAP binding/transport + soapAction, (b) service/port endpoint address.
2. Fault quote → status+body: e.g. SOAP soap:Fault (Client, UnknownCountry) → 404 {"error":{"code":"NOT_FOUND","message":"No tracking events found for UNKNOWN"}} (trackingEventService.js:33). 200-with-error breaks caching/retries/load-balancers (they treat it as success, cache poison, retry never fires).
3. UDDI publish/find/bind now: publish → git + Docker image + /api-docs/?wsdl serving; find → env-var + gateway routes + Swagger, not registry lookup; bind → build-time openapi.yaml/soap.createClientAsync(wsdlUrl) + setEndpoint, not runtime UDDI bind. Taken over by: container env, gateway routing, OpenAPI/Swagger, WSDL URL fetch.
4. XSD-before-code vs OpenAPI-not-enforced: function = validate(recordEventSchema) in services/tracking-service/src/routes/trackingRoutes.js:11 (AJV via @shiptrack/common). Without it, {"status":"NOT_A_STATUS"} or 500-char note would reach Mongoose and 500/poison data (test tracking.routes.test.js:28-33 would fail open).
5. Where SOAP still wins over REST: currency/country reference + any money movement needing exactly-once/ACID across orgs — buying WS-Security message-level signing/encryption + strict contract versioning, which REST+JWT+idempotency only approximates. "Nowhere" would need proof retries+idempotency+outbox already give same guarantees (they don't across trust boundaries).

How to run everything (copy-paste)

```
cd "/Users/nandankabra/Desktop/IIIT /sem 3 /webservices/shipment1212"
cp .env.example .env; docker compose up -d --build; docker compose ps  # 16 healthy
open http://localhost:3001  # frontend
open http://localhost:18000/api-docs  # gateway swagger
open http://localhost:8005/api-docs   # tracking swagger
open http://localhost:8011/graphql    # Apollo sandbox
curl -s http://localhost:8007/wsdl/country?wsdl | head -20
npm install; npm test --workspace services/tracking-service; npm test --workspaces --if-present
npx @redocly/cli lint services/tracking-service/openapi.yaml
```

Credentials (local dev only, from .env.example): admin@shiptrack.local / Admin@12345. DBs: Postgres localhost:15432, MySQL 13306, Mongo 27019 (see RUNBOOK.md).